My Little Environment

Burak Can Arikan

An IoT project based on Arduino UNO R4 WiFi, including a dashboard along with Telegram and Google Sheets integrations.

What is this project?

My Little Environment monitors a room's air and reacts when a value leaves the range you set.

It reads four values continuously:

- **Temperature** (°C)
- **Humidity** (%)
- **Carbon dioxide** (PPM)
- **Light** (brightness)

When a value is out of range the on-board light turns red, the buzzer can sound, and a Telegram message can be sent.

Connected features

- **Web dashboard:** opens the device's IP in a browser for live values, history graphs and settings.
- **Telegram bot:** requests a status report from anywhere and receive alerts in the chat.
- **Google Sheets:** each reading is logged to a spreadsheet for a long-term record.
- **OLED screen:** shows the current readings without a phone or computer.
- **Persistent memory:** settings and history survive a reboot.

Everything runs on one **Arduino UNO R4 WiFi** board.

Part	**Job**
DHT22 sensor	temperature and humidity
MQ-135 gas sensor	air quality / CO_2 estimate
LDR (photoresistor)	light level
SH1106 OLED screen	shows readings on the device
RGB LED	colour status (green / red / blue)
Buzzer	audible alert

The pin each part uses is kept in one place, `config.h`, so the wiring is easy to change.

Pins and settings live in config.h

All the wiring and account details sit in a single file. To use the project on your own network you only edit this one place.

```
#define DHT_PIN     4      // temperature + humidity
#define MQ135_PIN   A0     // air quality
#define LDR_PIN     A1     // light
#define BUZZER_PIN  7
#define LED_R 9
#define LED_G 10
#define LED_B 11

#define WIFI_SSID  "your-wifi-name"
#define WIFI_PASS  "your-wifi-password"
#define BOT_TOKEN  "your-telegram-bot-token"
#define CHAT_ID    "your-telegram-chat-id"
#define SHEETS_PATH "https://script.google.com/macros/s/.../exec"
```

(Replace the placeholders with your own values before uploading.)

How the code is organised

The project is split into small files, each with one clear job.

`my_little_environment.ino`	start-up and the main loop
`config.h`	pins, wifi, tokens, constants
`monitor.h`	shared data structures and function list
`sensors.cpp`	reading the sensors, CO_2 calibration
`alerts.cpp`	range checks, LED, buzzer, notifications
`display.cpp`	drawing on the OLED screen
`history.cpp`	clock, history buffers, saving to memory
`network.cpp`	WiFi, Telegram, Google Sheets
`webserver.cpp`	the web server and JSON data feed
`dashboard.cpp`	the dashboard web page itself

The heart: the main loop

The loop checks the clock and runs each job once its interval has passed.

```
void loop() {
  unsigned long now = millis();

  if (now - lastRead >= 2000) {        // every 2 seconds
    lastRead = now;
    readSensors();
    drawScreen();
    updateAlerts();
  }

  if (now - lastTelegram >= 5000)     checkTelegram();   // every 5 s
  if (now - lastSheets   >= 300000UL) logToSheets();     // every 5 min
  if (now - lastSave     >= 1800000UL) saveState();      // every 30 min

  handleWeb();                         // answer browser requests
}
```

Reading the sensors

Temperature and humidity come from the DHT22. The MQ-135 gives a raw number that is turned into a CO_2 value with a known formula.

```
void readSensors() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  if (!isnan(t)) readings[TEMP].value  = t;
  if (!isnan(h)) readings[HUMID].value = h;
  readings[CO2].value   = ppmFromRaw(analogRead(MQ135_PIN));
  readings[LIGHT].value = analogRead(LDR_PIN);
}
```

The gas sensor is calibrated once in fresh air (`calibrate()`), which sets the clean-air baseline `R0` such that the readings stay accurate.

Deciding when something is wrong

Each reading is compared with the desired range. The status LED reflects the result at a glance: red when out of range, blue at night, green when all is well.

```
void updateAlerts() {
  bool out = !inRange(TEMP) || !inRange(HUMID) || !inRange(CO2);
  if (out)          setLED(255, 0, 0);   // red:   out of range
  else if (dark())  setLED(0, 0, 255);   // blue:  dark room
  else              setLED(0, 255, 0);   // green: all good

  String tele = alertReason(settings.telegram);
  if (tele.length() && !telegramOn)
    sendTelegram("Notification: " + tele + ".");
  telegramOn = tele.length() > 0;
}
```

The `telegramOn` flag makes the message fire once, when a reading first goes out of range, rather than on every cycle.

Remembering history

The device keeps three rolling histories at different zoom levels:

Series	Sample every	Covers
Hour	1 minute	last hour
Day	15 minutes	last day
Week	2 hours	last week

Each one is a fixed-size ring: when it fills up, the oldest sample is overwritten. This means memory use never grows.

Every 30 minutes the settings and all three histories are written to the board's EEPROM, so a power cut does not erase them. On start-up the device loads them back.

Saving and restoring state

A small signature is stored first. On boot, if it matches, the saved data is trusted and loaded; if not, the device starts fresh.

```
void saveState() {
  int a = 0;
  eePut(a, STATE_SIGNATURE);     // marker that says "valid data here"
  eePut(a, settings);            // all your chosen ranges
  for (int s = 0; s < SERIES_COUNT; s++) {
    eePut(a, series[s].head);
    eePut(a, series[s].fill);
    eePut(a, series[s].times);
    eePut(a, series[s].values);  // the history graphs
  }
}
```

The web dashboard

The device runs its own tiny web server. The full dashboard page (HTML, style and graphs) is stored on the board and served on request.

```
void handleWeb() {
  WiFiClient client = server.available();
  if (!client) return;
  // ... read the request ...

  if (req.indexOf("GET /calibrate") >= 0) { calibrate(); sendOk(client); return; }
  if (req.indexOf("GET /set/")      >= 0) { /* change a setting */ return; }
  if (req.indexOf("GET /api")       >= 0) { sendApi(client, req); return; }

  sendDashboard(client);   // otherwise: send the web page
}
```

The page asks `/api` for fresh numbers every 3 seconds and redraws the live graphs in the browser.

What the dashboard shows

Open `http://<device-ip>` in a browser and you get:

- the current temperature, humidity, CO_2 and light, each marked as fine or out of range
- history graphs you can zoom to the last hour, day or week, or any custom time window, with the average drawn as a dashed line
- input boxes to set your desired ranges (too cold, too warm, too humid, ...)
- checkboxes to choose, per reading, whether the buzzer and/or Telegram should warn you
- a button to calibrate the air sensor in fresh air.

Telegram from anywhere

The device talks to Telegram over a secure connection. It can send you a message, and it also listens for commands you type in the chat.

```
static void handleCommand(String command) {
  if (command == "/status")    sendTelegram(statusText());
  else if (command == "/calibrate") {
    calibrate();
    sendTelegram("Calibrated. R0 = " + String(settings.r0, 2));
  }
  else if (command == "/help") sendTelegram(helpText());
  else sendTelegram("Unknown command. Send /help.");
}
```

`/status` returns the current readings, `/calibrate` re-runs the fresh-air calibration, `/help` lists the commands.

Logging to Google Sheets

Every five minutes the readings are sent to a Google Apps Script web app, which appends a new row to your spreadsheet. This builds a long history in the cloud.

```
void logToSheets() {
  httpsGet(SHEETS_HOST,
    String(SHEETS_PATH) +
      "?temp="  + String(readings[TEMP].value, 1) +
      "&humid=" + String(readings[HUMID].value, 1) +
      "&co2="   + String(readings[CO2].value, 0) +
      "&light=" + String(readings[LIGHT].value, 0),
    false);
}
```

The values are passed in the web address; the Apps Script reads them and writes them into the sheet.

What you need to run it

Hardware

- Arduino UNO R4 WiFi
- DHT22, MQ-135, an LDR, an SH1106 OLED, an RGB LED, a buzzer
- wired to the pins listed in `config.h`

Software (Arduino IDE)

- Board package: *Arduino UNO R4 Boards*
- Libraries: *DHT sensor library* (+ *Adafruit Unified Sensor*) and *U8g2*
- WiFi, Wire and EEPROM come built in with the board

How to run it – step by step

1. Place all project files in the `my_little_environment` folder and open the `.ino` file in the Arduino IDE.
2. Open `config.h` and enter your WiFi name and password. Optional: Include your Telegram bot token, chat id, and Google Sheets web-app link.
3. Select *Arduino UNO R4 WiFi* and the appropriate port, then press **Upload**.
4. Launch the Serial Monitor at **115200** baud. After connecting, it prints the following line:
   `Dashboard: http://192.168.x.x`item
   Go to that address in a browser; the dashboard is now operational. To calibrate the sensor, first leave it in fresh air and press *Calibrate* once.

Setting up Telegram and Google Sheets

Telegram (optional)

- Get your chat id and bot id; put them the into `config.h`.
- You can then use `/status`, `/calibrate`, `/help` in the chat.

Google Sheets (optional)

- Make a Google Sheet, open *Extensions/Apps Script*.
- Write a short script that reads the values from the web address and adds a row.
- *Deploy* it as a web app and copy the `/exec` link into `SHEETS_PATH`.